

TERM	COURSE NAME	COURSE CODE AND SECTION	VERSION
2257	Introduction to Programming using C	IPC144 - NAA	A

Student Name	
Student Number	

PROFESSOR	Catherine Leung
DATE OF FINAL ASSESSMENT	December 11
DURATION OF FINAL ASSESSMENT	2 hours
WEIGHT OF FINAL ASSESSMENT	35%

Mark breakdown:

PART A: Concepts and Debugging	10 marks
PART B: Walkthrough	20 marks
PART C: Programming	20 marks
TOTAL MARKS	50 marks

SPECIAL INSTRUCTIONS:

1. Mobile phones, smartwatches, earbuds and all other electronic need to be put in bags and placed under your chair or at the front or back of the room.
2. Answer all questions in the space provided on the test paper.
3. Communication is only allowed with your professor or lab monitor. There is to be no communication with other students.

This exam includes this *cover page*, plus (9) pages of questions.

SENECA'S ACADEMIC INTEGRITY POLICY

As a Seneca student, you must conduct yourself in an honest and trustworthy manner in all aspects of your academic career. A dishonest attempt to obtain an academic advantage is considered an offense and will not be tolerated by the Polytechnic.

By beginning this test, I affirm that I will not give or receive any unauthorised help in this test and that all work provided will be my own. I agree to abide by Seneca's Academic Integrity Policy, and I understand that any violation of academic integrity will be subject to the penalties outlined in the policy.

Question 1 [10 marks]: Debugging

The following program uses an array of **Pair** structs and determines the sum of the highest number in each **Pair** by calling a helper function **sumHighest()** which returns the sum which is then displayed.

However, there are bugs in the code below.

- a) Identify all the bugs that you can find. Mark each bug as (S) for syntactic, (L) for logical. (6 marks)
 b) provide a fix for the code by crossing out the errors and writing the fixes where they apply (4 marks)

```

1. #include <stdio.h>
2. #include <string.h>
3. #define MAX 10
4.
5. struct Pair
6. {
7.     int num1;
8.     int num2;
9. };
10. int sumHighest(struct Pair array[]);
11.
12. int main(void)
13. {
14.     struct Pair numbers[MAX] = {{5, 1}, {2, 6}, {1,4}, {3,5}};
15.     int result = sumHighest(numbers, MAX);
16.     printf("sum of biggest is: %d\n", result);
17.     return 0;
18. }
19. int sumHighest(struct Pair array[])
20. {
21.     int i;
22.     int rc;
23.     int len = strlen(array);
24.     for (i = 1; i <= len; i++){
25.         if (array[i] < array[i+1]){
26.             rc += array[i];
27.         }
28.         else{
29.             rc += array[i+1];
30.         }
31.     }
32.     return rc;
33. }

```

Question 2 [20 marks total]: Programming

You are working on an application to help a runner track their performance. Your program keeps track of each running session and provides summaries to the user. The user tracks their running sessions in a file. The file has a header row followed by a record on each line. Note this is just an example of what the file could look like. Your program must work to spec, not just for this set of data.

```
day-month-year;distance in kilometers;time in seconds; outdoor temperature
2-1-2025;3.1;1846;-7
30-12-2024;2.1;1200;-5
1-1-2025;2.3;1206;-6
23-3-2025;2.0;1156;4
```

Part A: [3 marks]

Create a struct that can store a row of data from the file called Session. Use the example to determine the data type of each of the data in the struct. Tip: you may wish to read the later questions before deciding how to proceed.

```
struct Session
{
```

```
}
```

Part B: [4 marks]

Write the following function. This function accepts the name of the file (**filename**) containing the runner's information as described in Part A. It is also passed an array of Session structs (**data**) and an integer (**max**) indicating the maximum number of records that can be stored into the array.

This function opens the file and reads the records from the file into the array and returns the number of records read. As the array can only store max number of records, only max records should be read at most.

```
int readSessionFile(const char filename[], struct Session data[], int max)
{
```

```
}
```

Part C: [4 marks]

Write the following function. This function accepts an array of Session structs (**data**), the number of records stored in the array(**used**) and a particular month (**month** and **year**). This function calculates the average number of minutes per kilometer during the given month. The average is calculated by summing the total time and dividing it by the total distance.

```
double getAverageMinPerKm(struct Session data[], int used, int month, int year)
{
```

```
}
```

Part D: [4 marks]

Write the following function. This function accepts an array of Session structs (**data**), and the number of records in the array(**used**). This function returns the index of the record with the earliest date.

```
int earliestRecord(struct Session data[], int used);
```

Part D: [5 marks]

Write the following function. This function accepts an array of Session structs (**data**), the number of records stored in that array (**used**) and 3 pointers to integers (**cold, warm, hot**). The function counts the number of records within 3 categories: cold days, warm days and hot days and passes that information back through its corresponding pointers. The following is used to categorize the temperature each day:

- cold - below 10 C
- warm - between 10 C and 25 C inclusive
- hot - over 25 C

```
void countDays(struct Session data[], int used, int* cold, int* warm, int* hot)
{
```

```
}
```

Part F: [5 marks]

Write a program that will make use of the functions you wrote previously. You will get full marks for this question if you can demonstrate how to use the functions even if the functions themselves have bugs.

This program will use the data stored in runner.txt. This file is formatted according to the description in part A. It will ask the user to enter a month and a year. You may assume that the data is valid. You do not need to error check. The program will then print a report that prints the starting date in the data set, the number of cold/warm/hot days on record, average minutes per km for the given month. Here is a sample run. Note that anything that is **bolded** is user entered. Anything in angle brackets <> is data that you will need to obtain programmatically.

```
The Running App

Enter a month: 1
Enter a year: 2025

Running Report – from <earliest date> to present

Average Minutes per KM: <min per km>

Number of days ran
...in the cold: <number of cold days>
...in warm weather: <number of warm days>
...in hot weather: <number of hot days>
```


Question 3 [15 marks]: Walkthrough

What is the exact output of the following your program? Exact full output [10 marks], tables/diagrams/etc. [5 marks]

```

1.  #include <stdio.h>
2.  #include <string.h>
3.  #include <ctype.h>
4.  #define MAX 6
5.  void chooseThis(int apple, int* leaf, int* petal);
6.  void doThat(char final[], const int data[], int used);
7.  int main(void)
8.  {
9.      int data[MAX] = { 8167, 7219, 6235, 3519, 6327, 1243};
10.     char final[11] = "";
11.     doThat( final,data, MAX);
12.     printf("Final: %s\n", final);
13.     return 0;
14. }
15. void doThat(char final[], const int data[], int used){
16.     int i, j;
17.     int first, second, third;
18.     char cream[3]="GG";
19.     char sugar[11]="";
20.
21.     for (i = 0;i<used;i++){
22.         chooseThis(data[i], &first, &second);
23.         third = (first + second) % 10;
24.         printf("(%d + %d) %% 10 = %d\n", first, second, third);
25.         switch(third){
26.             case 1:
27.                 strcat(final, "co");
28.                 break;
29.             case 2:
30.                 strcat(sugar,"ca");
31.                 break;
32.             case 3:
33.                 if(strcmp(cream,"GG")==0){
34.                     strcat(final, "ke");
35.                 }
36.                 else{
37.                     strcat(sugar,"ke");
38.                 }
39.                 break;
40.             case 4:
41.                 strcat(final,sugar);
42.                 break;
43.             default:
44.                 for(j = 0; cream[j]!='\0'; j++){
45.                     cream[j] = tolower(cream[j]) - 1;
46.                 }
47.                 strcat(sugar,cream);
48.             }
49.             printf("%s\n", final);
50.             printf("%s\n", sugar);
51.         }
52.     }
53. void chooseThis(int apple, int* leaf, int* petal){
54.     *leaf = apple % 10;
55.     *petal = apple/1000;
56. }

```

Put the exact output for this program in this box:

This page is for rough work